BEGINNING OF TRANSMISSION

State: unpublished / pending pairing
Pairing: pending — unresolved: - GitHub release URL: `pending` - PDF SHA-256: `pending`

Release metadata

Field	Value
Title	Active Inference Multi-Track Exemplar
Version	0.3.0
Concept DOI	10.5281/zenodo.20417021
Version DOI	10.5281/zenodo.20420352
GitHub	docxology/template_active_inference
Zenodo	https://zenodo.org/records/20417021
SHA-256	pending
SHA-512	pending

How to verify

- Scan **Integrity** QR and compare the embedded SHA-256 prefix to the table above.
- Scan **Zenodo** / **GitHub** QR codes and confirm they resolve to this release pairing.
- Full hashes and structured fields: `../data/transmission_manifest.json`.

Figure 1: Integrity QR strip

Structured manifest: `../data/transmission_manifest.json`

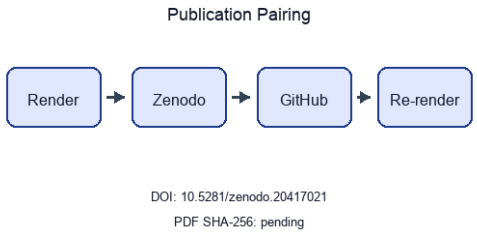

Figure 2: Publication pairing flow

Stego: off | overlays text | barcodes on | XMP on | manifest on → `./secure_run.sh`

Active Inference Multi-Track Exemplar

Sheaf-Composed Manuscript with pymdp Sophisticated Inference

Daniel Ari Friedman
Active Inference Institute
daniel@activeinference.institute
ORCID: 0000-0001-6232-9096
DOI: 10.5281/zenodo.20417021

May 30, 2026

Contents

1 Sheaf Track Coverage

This page summarizes which **sheaf fragment tracks** are bound for each IMRAD row in `manuscript/sheaf/manifest.yaml`. The matrix is regenerated at compose time.

Totals: 53 present / 53 bound / 0 missing (gray).

Color	Meaning
Black	Track **present** (bound and fragment exists)
White	**Absent** (not bound for this row)
Gray	**Missing** (bound but fragment file absent)

1.1 Introduction

- **Introduction** *(group)*
- **Motivation and scope**
- **Contributions** ## Methods
- **Methods** *(group)*
- **Bernoulli–Ising analytical model**
- **pymdp simulation harness**
- **Lean formalization boundary**
- **Sheaf composition** ## Results
- **Results** *(group)*
- **Mutual-information parameter sweep**
- **Free-energy decomposition**
- **T-maze active-inference rollout**
- **Validation invariants** ## Discussion
- **Discussion** *(group)*
- **Limitations and outlook** ## Appendix
- **Appendix** *(group)*
- **Appendix: full track coverage**

Coverage overview. Sheaf track coverage matrix: 17 IMRAD rows × 13 fragment columns. Black = present (P), white = absent (—), gray = missing (M). Counts: 53 present / 53 bound / 0 missing.

Appendix row `16_appendix_full_sheaf.md` binds 12 fragment track types as a composability proof (registry defines 13 types; optional `layers` is methods-only).

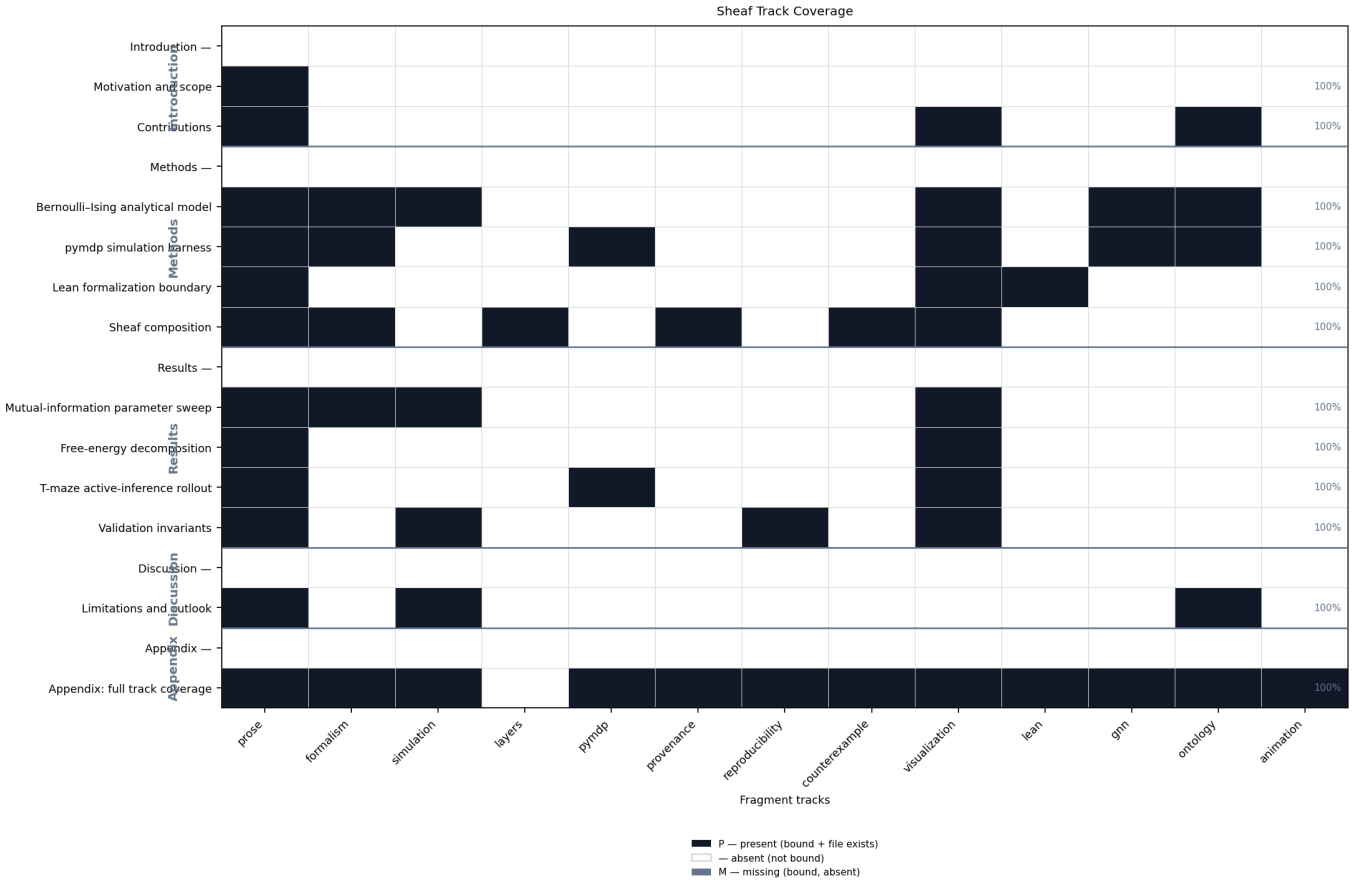

Figure 3: Heatmap matrix of IMRAD manuscript rows versus 13 sheaf fragment track columns. Black cells mean the track is bound and the fragment file exists; white cells mean the track is not bound; gray cells mean bound but missing. Rows are grouped by IMRAD block with indented subsection labels; column headers list track ids.

2 Abstract

We study a minimal Active Inference stack on toy models: a Bernoulli–Ising analytical oracle, a pymdp T-maze rollout, and a sheaf-indexed compose contract that binds 13 fragment tracks into 12 flat IMRAD sections. Claims are limited to those models and their generated artifacts.

sec. 1 reports a 17-row coverage matrix (5 IMRAD group headers) regenerated from the live manifest at compose time. sec. 6 documents the T-maze harness aligned with pymdp sophisticated_inference examples.

sec. 12 records 12 / 12 invariant checks passed. SI planning horizon: 2 steps. Sweep RMSE 0 nats bounds analytical–empirical agreement on the coupling grid.

Introduction

3 Motivation and scope

3.1 Scientific scope

This manuscript couples three tracks on toy Active Inference models: a Bernoulli–Ising analytical oracle, a pymdp T-maze rollout, and a sheaf-indexed assembly contract that binds 13 optional fragment tracks under an IMRAD outline. The scientific claims stay within those models and their generated artifacts; they are not empirical statements about biological agents.

3.2 Manuscript structure

Three **scientific tracks** (analytical, pymdp, sheaf composition) map onto 13 **composable fragment types** and 10 pipeline gates (fig. 4). sec. 1 summarizes which fragment tracks bind to each manifest row. sec. 8 documents the compose pipeline, coverage semantics (eq. 3), and strict validation gates.

The pymdp track follows the pymdp sophisticated_inference examples with a minimal T-maze and planning horizon `policy_len = 2`. Other sections cite sec. 6 instead of repeating that reference.

4 Contributions

4.1 Scientific contributions

1. **Analytical oracle** (sec. 5): closed-form mutual information and free-energy decomposition on a symmetric Bernoulli–Ising toy with Monte Carlo cross-checks (sec. 9, sec. 10).
2. **Active-inference harness** (sec. 6): deterministic pymdp T-maze rollout — default `state_inference` belief filtering, with sophisticated expected-free-energy policy inference selectable via `mode: policy_inference` — with logged beliefs, actions, and merged invariant gates (sec. 11, sec. 12).
3. **Sheaf-indexed composition** (sec. 8): 13 optional fragment types bind to 17 manifest rows under eq. 3, with a 12-track appendix composability proof (sec. 14).

fig. 4 maps the three scientific tracks to 10 pipeline gates and 13 composable fragment renderers. Measured invariant checks: 12 / 12 passed.

Ontology-facing symbols are checked per model: the Bernoulli toy binds `pi1`, `pi2`, `J`, `gamma`, and `q_joint`, while the SI T-maze binds `location`, `observation`, `policy`, and `belief_entropy` to **HiddenState**, **ObservationLikelihood**, **PolicyPosterior**, and **BeliefEntropy** (fig. 6, sec. 6).

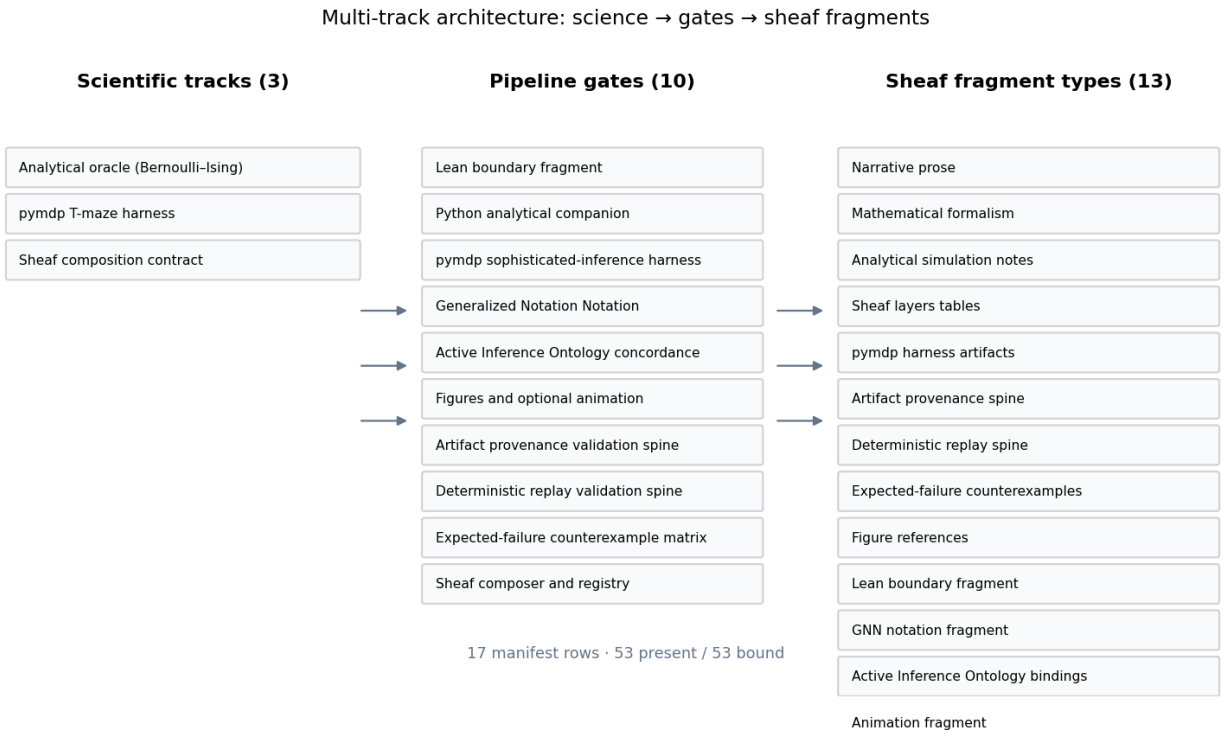

Figure 4: Process diagram linking three scientific tracks to 10 pipeline gates and 13 sheaf fragment types across 17 manifest rows.

Figure I1 (intro). Multi-track architecture: analytical, pymdp, and sheaf composition lanes mapped to 10 pipeline gates and 13 composable fragment types.

4.1.1 Ontology bindings

- `expected_free_energy` → **ExpectedFreeEnergy**
- `location` → **HiddenState**
- `observation` → **ObservationLikelihood**
- `policy` → **PolicyPosterior**

Methods

5 Bernoulli–Ising analytical model

We study a minimal **K=2 Bernoulli / Ising** coupling as the analytical companion to multi-track verification. The entangled joint eq. 1 yields closed-form mutual information $I(\lambda)$, which serves as an oracle for Monte Carlo checks and GNN round-trips in sec. 9 (fig. 6).

Measured sweep grid points: 21. Invariants passed: 12 / 12.

The entangled joint over binary policies satisfies

$$q_\lambda(\pi) \propto E(\pi)\, \exp(\lambda J(\pi)), \tag{1}$$

with symmetric Ising coupling J and deformation parameter λ. Mutual information is $I(\lambda) = \log 2 - H_b(\sigma(\lambda))$.

The analytical track writes a parameter sweep comparing closed-form and empirical mutual information across $\lambda \in [0, 4]$ on 21 grid points (sec. 9, fig. 5).

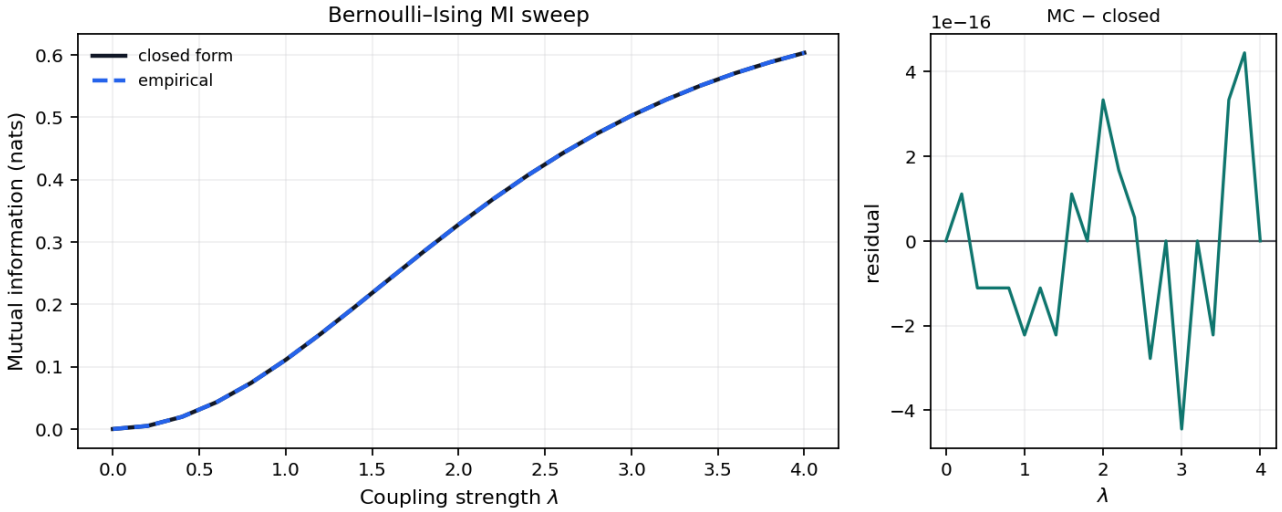

Figure 5: Two-panel plot of mutual information versus coupling strength lambda for the symmetric Bernoulli-Ising toy. Left panel shows closed-form curve (solid dark line) and Monte Carlo estimates (dashed blue line) with lambda on the x-axis and mutual information in nats on the y-axis. Right panel shows Monte Carlo minus closed-form residuals versus lambda with a zero reference line.

Figure 1 (methods). Closed-form and Monte Carlo mutual information I(λ) for the symmetric Bernoulli-Ising toy across 21 grid points up to λ_max = 4; grid maximum 0.6031 nats on the measured sweep.

Figure 1b (methods). GNN ↔ ontology concordance for the Bernoulli–Ising toy (GNN v1.1).

The Bernoulli toy is declared in `gnn/bernoulli_toy.gnn.md` (GNN v1.1). fig. 6 links GNN variables to Active Inference Ontology terms bound in the analytical ontology fragment; round-trip parity is checked before render.

Measured MI and sweep artifacts in sec. 9 ground the same symbol map used in the concordance diagram.

5.0.1 Ontology bindings

- `E1` → **Stream1HabitPrior**
- `E2` → **Stream2HabitPrior**
- `J` → **CrossStreamCouplingPotential**
- `gamma` → **SophisticationWeight**
- `lam` → **EntanglementDeformationParameter**
- `pi1` → **Stream1PolicyVector**
- `pi2` → **Stream2PolicyVector**
- `q_joint` → **EntangledJointPosterior**

Figure 6: Layered concordance diagram linking analytical symbols, GNN variables from bernoulli_toy.gnn.md (GNN v1.1), and Active Inference Ontology terms.

6 pymdp simulation harness

Sophisticated inference (planning horizon). This section documents a **pymdp state-inference harness** on a minimal T-maze (fig. 7) with planning horizon `policy_len = 2`. Default mode is `state_inference` (T-maze rollout via `simulate_si_tmaze.py`). The Agent constructs a multi-step policy set (`num_policies` logged in SI artifacts); per-step **belief entropy** is aggregated as `mean_belief_entropy`. Selecting `mode: policy_inference` enables expected-free-energy policy selection; on the default minimal T-maze (2 states / 2 observations / 2 actions, horizon 2) this yields a constant-action policy that does not reach the goal — the toy is deliberately too small to exercise sophisticated inference's advantage, so richer state spaces and longer horizons are needed for it to differ from belief filtering.

Graph-world artifacts are deterministic extension outputs declared in `tracks.yaml extension_tracks.graph_world`. For the reference workflow, see sec. 3; measured rollouts appear in sec. 11.

Mean belief entropy across steps: 0.3251.

The comparison artifact `output/data/si_policy_comparison.json` runs both `state_inference` and `policy_inference` over the declared toy horizons and seeds without replacing the default rollout. It records 4 deterministic comparison rows, of which 3 reach the goal under the toy transition model.

The graph-world extension is deterministic: `simulate_si_graph_world.py` writes `si_graph_world_summary.json` and `si_graph_world_trace.json` for a four-node graph-world path. The regenerated summary reports 4 nodes, 4 steps, and goal-reached flag 1.

Given generative matrices A, B, C, D, pymdp computes state beliefs $q(s)$ via variational inference (`infer_states`). The Agent is configured with planning horizon $H = 2$, which defines the **policy depth** used when constructing candidate policies (logged as `num_policies` in the SI summary artifact; see sec. 11).

The default harness records belief entropy per step; extending to full expected-free-energy policy selection (`infer_policies`) is documented as a follow-on track in sec. 13.

SI artifacts (summary, trace, optional JSONL log) record step count, actions, observations, and belief entropy for sec. 11. Steps recorded: 2.

Figure M1 (methods). T-maze generative model schematic (2-step policy horizon, state_inference mode).

See `gnn/si_tmaze.gnn.md` for a GNN view of the T-maze hidden state, observation, and policy variables with ontology bindings.

6.0.1 Ontology bindings

- `belief_entropy` → **BeliefEntropy**
- `loc` → **HiddenState**
- `obs` → **ObservationLikelihood**
- `pi` → **PolicyPosterior**

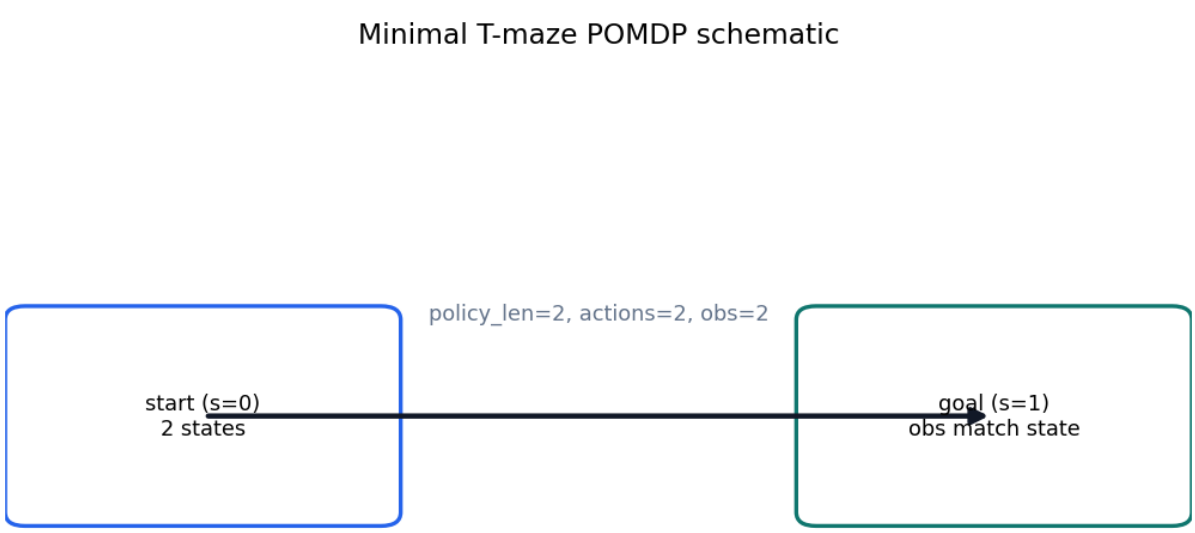

Figure 7: Schematic of the minimal T-maze POMDP with start and goal states, discrete actions and observations, and planning horizon policy_len = 2.

7 Lean formalization boundary

The Lean track provides minimal boundary witnesses checked by `lake build` under `lean/TemplateActiveInference/`. fig. 8 summarizes proved versus deferred statements; fragments cite theorem names without duplicating proof scripts in prose.

Horizon witnesses link back to the analytical toy (sec. 5) and the pymdp planning depth (sec. 6).

Figure M2 (methods). Lean formalization boundary: module witnesses checked by lake build.

Lean module `TemplateActiveInference.SophisticatedInference` declares the planning-horizon parameter `defaultPolicyLen` and finite T-maze boundary witnesses: `sophisticated_requires_horizon : defaultPolicyLen > 1`, `tmaze_two_forward_steps_reach_goal`, and `tmaze_goal_absorbing`. These theorems formalize the small state-transition boundary shared with the Python harness; they do *not* prove that the toy policy posterior is a general model of sophisticated inference. Axioms are audited with `#print axioms` (the gate whitelists only `propext`, `Classical.choice`, `Quot.sound`); see the Lean track gate.

Build via `lake build` under `lean/`.

Lean formalization boundary status

Module	Kind	Name	Status
TemplateActiveInference.Bernoul	def	isingCouplingEntries	proved
TemplateActiveInference.Bernoul	def	isingCouplingSum	proved
TemplateActiveInference.Bernoul	theorem	ising_coupling_sum_zero	proved
TemplateActiveInference.Sophisti	def	defaultPolicyLen	proved
TemplateActiveInference.Sophisti	theorem	sophisticated_requires_horizon	proved
TemplateActiveInference.Sophisti	def	tmazeStep	proved
TemplateActiveInference.Sophisti	theorem	tmaze_two_forward_steps_reach_	proved
TemplateActiveInference.Sophisti	theorem	tmaze_goal_absorbing	proved
TemplateActiveInference.Sophisti	def	graphWorldStep	proved
TemplateActiveInference.Sophisti	theorem	graph_world_three_steps_reach_	proved
TemplateActiveInference.Sophisti	def	finitePolicies	proved
TemplateActiveInference.Sophisti	theorem	policy_enumeration_contains_for	proved

Figure 8: Table figure listing Lean modules, declaration kinds, names, and proved versus sorry status under lean/TemplateActiveInference/.

8 Sheaf composition

8.1 Compose contract

Each manifest row in `manuscript/sheaf/manifest.yaml` binds fragment tracks from `manuscript/sheaf/tracks.yaml`. A track supplies a renderer, compose order, label, and optional flag; the composer flattens the binding set into one Markdown section for PDF and web output.

The operational claim is auditable binding: analytical, simulation, pymdp, visualization, Lean, GNN, ontology, and optional media fragments attach to each IMRAD row under eq. 3 (**P** present, — unbound, **M** missing).

8.2 Coverage and figures

fig. 9 summarizes 13 fragment types and their IMRAD bindings. Generated tables below list every track definition and section×track binding at compose time.

8.3 Compose commands

```
uv run python scripts/compose_manuscript.py
uv run python scripts/compose_manuscript.py --validate-only --strict
```

Each run emits `output/data/sheaf_coverage_matrix.json` and regenerates coverage artifacts. Partial compose (`--section`) is draft-only; the matrix always reflects the full manifest. Coverage totals appear on sec. 1; discussion scope is in sec. 13.

8.4 Law verification

`--validate-only --strict` runs the structural gate before any fragment is glued. Beyond per-cell coverage, it invokes the sheaf-law oracle (`verify_sheaf_laws`, `src/manuscript/sheaf/laws.py`), which checks 6 axioms — poset, presheaf functoriality, separation, gluing, typing, and compositionality — and reports 6/6 satisfied for the current manifest. A violation is raised as an error-level issue and aborts the build, so a malformed manifest (a section colliding on an output file, an off-chain block, a mistyped fragment, a fragment shared between sections) can never compose. The formal statements are in the formalism block below; the negative-control suite (`tests/test_sheaf_laws.py`) proves each check is falsifiable.

The semantic layer is separate from those structural laws. `output/data/sheaf_gluing_certificate.json` records cross-track symbols, typed claim evidence, artifact sources, and manuscript-variable restrictions; validation fails when the analytical, pymdp, GNN, ontology, Lean, visualization, or manuscript tracks disagree about a shared symbol or measured claim.

8.4.1 Base poset and presheaf

The manuscript is modelled as a coverage sheaf over a finite base poset. Let the **base** P be the IMRAD blocks ordered as a chain,

$$\mathsf{Introduction} \prec \mathsf{Methods} \prec \mathsf{Results} \prec \mathsf{Discussion} \prec \mathsf{Appendix}, \tag{2}$$

with, in each block, a *group* node above its *section* nodes (written $G \sqsupseteq s$). P is therefore a finite poset (equivalently a finite Alexandrov space). Let $\mathcal{T}$ be the registered fragment-track set from `manuscript/sheaf/tracks.yaml`; each track $t \in \mathcal{T}$ carries a renderer $R(t)$, label $L(t)$, optional flag $O(t)$, and a strict compose-order index $\pi(t)$.

The **presheaf** $\mathcal{F}$ is a contravariant functor on P — $\mathcal{F} : P \to \mathbf{Set}$ with restriction maps along $\sqsupseteq$ — assigning to each composing section s its bound fragment set $\mathcal{F}(s) = \{\,(t, F_s(t)) : t \text{ bound in } s\,\}$, where $F_s : \mathcal{T} \rightharpoonup \mathbf{Path}$ is the section's partial binding map. Restriction along $G \sqsupseteq s$ is projection onto a section's own bindings; group nodes carry the empty assignment and do not compose.

The coverage cell is

$$B(s,t) \in \{\mathrm{P}, —, \mathrm{M}\} \tag{3}$$

derived from $F_s(t)$ and filesystem existence at compose time: **P** when a bound fragment exists, — when the track is unbound for that row, and **M** when a bound path is missing. The current regenerated matrix reports 53 present / 53 bound / 0 missing cells. Registry size: $|\mathcal{T}| = 13$ types across 17 IMRAD manifest rows (5 group rows, 12 composing sections).

8.4.2 Verified sheaf laws

What makes this presheaf a *sheaf* — rather than a bare incidence table — is that the composer's structural axioms are machine-checked. The oracle `verify_sheaf_laws` (`src/manuscript/sheaf/laws.py`) verifies 6 laws, and the regenerated build reports 6/6 satisfied:

1. **Poset.** The IMRAD blocks form the chain of eq. 2; compose order is monotone in block rank and every composing section's block carries a group row.
2. **Presheaf (functoriality).** Every bound track lies in $\mathcal{T}$; π is a strict total order; and each section's resolved track order is the monotone restriction of π (an explicit `track_order` override must be a permutation of the section's bound tracks).
3. **Separation (locality).** The map $s \mapsto$ output_name(s) is injective over composing sections: distinct locals glue to distinct global positions, so the global section is unique.
4. **Gluing.** Compose order is a linear extension of P — each block's rows are contiguous and strictly increasing in order — so the local fragments glue to a unique global manuscript in which every composing section appears exactly once.
5. **Typing.** Each binding $(t, F_s(t))$ is well-typed: $R(t)$ is a registered renderer and the fragment suffix lies in $R(t)$'s accepted suffix set. Generated renderers (`section_figures`, `layers_report`) synthesize their body and are explicitly type-exempt.
6. **Compositionality.** Every fragment file is private to one section (no path is bound twice), so global composition is the coproduct of the per-section bodies and is independent of inclusion order.

Each law is paired with a negative control in `tests/test_sheaf_laws.py` — a single mutation that breaks the law and is proven to be caught — so the gate binds the laws' *content*, not merely their shape. Under `--strict`, any violation is surfaced as an error-level manifest issue and aborts composition.

8.4.3 Scope (what is and is not claimed)

These laws verify the sheaf *axioms* on a finite base poset. They do **not** compute sheaf *cohomology* (H^0/H^1, Čech complexes, derived functors); "sheaf" here names the verified separation-and-gluing structure of a multi-track coverage assignment, not a cohomological invariant. Formal track definitions and section×track bindings appear in the generated tables below.

Semantic gluing then checks agreement of the glued content: coverage counts, manuscript variables, typed claim predicates, pymdp mode/hash, Bernoulli GNN ontology, and SI T-maze GNN ontology. This certificate is a content-level audit over the same base, not an additional topological law.

Figure 6 (methods). Sheaf layers overview: registry stack (compose order, renderer ids) and IMRAD binding heatmap for 13 fragment types across 17 manifest rows (53 present / 53 bound / 0 missing).

Figure 6b (methods). Semantic gluing graph: configured producers, generated evidence artifacts, and validation consumers for the multi-track sheaf certificate.

The `provenance` fragment makes artifact lineage a live validation-spine track. The configured producer `generate_validation_spine.py` writes `output/data/artifact_provenance.json`, which hashes 14 core artifacts and records their producer scripts plus config digests. Publication claims that depend on generated files must be traceable to this lineage table or to a narrower artifact-specific certificate.

The first promoted provenance claim is intentionally limited: every listed core artifact exists, has a SHA-256 digest, and is produced by a configured analysis script. A changed file, missing producer, or stale saved digest is a validation failure, not a prose warning.

The `counterexample` fragment records expected-failure fixtures as first-class evidence. `output/reports/counterexample_matrix.json` lists 5 negative controls that intentionally mutate ontology mappings, semantic certificates, graph-world trace agreement, typed claim evidence, and provenance hashes.

The matrix is not an empirical result. It is a falsifiability ledger: each row names the gate that must fail and the test that proves the failure path remains live.

Figure 9: Two-panel overview of sheaf fragment layers. Left panel shows 13 composable track types in registry compose order with labels and renderer ids. Right panel shows the IMRAD section binding heatmap with black present, white absent, and gray missing cells across 17 manifest rows and 13 tracks.

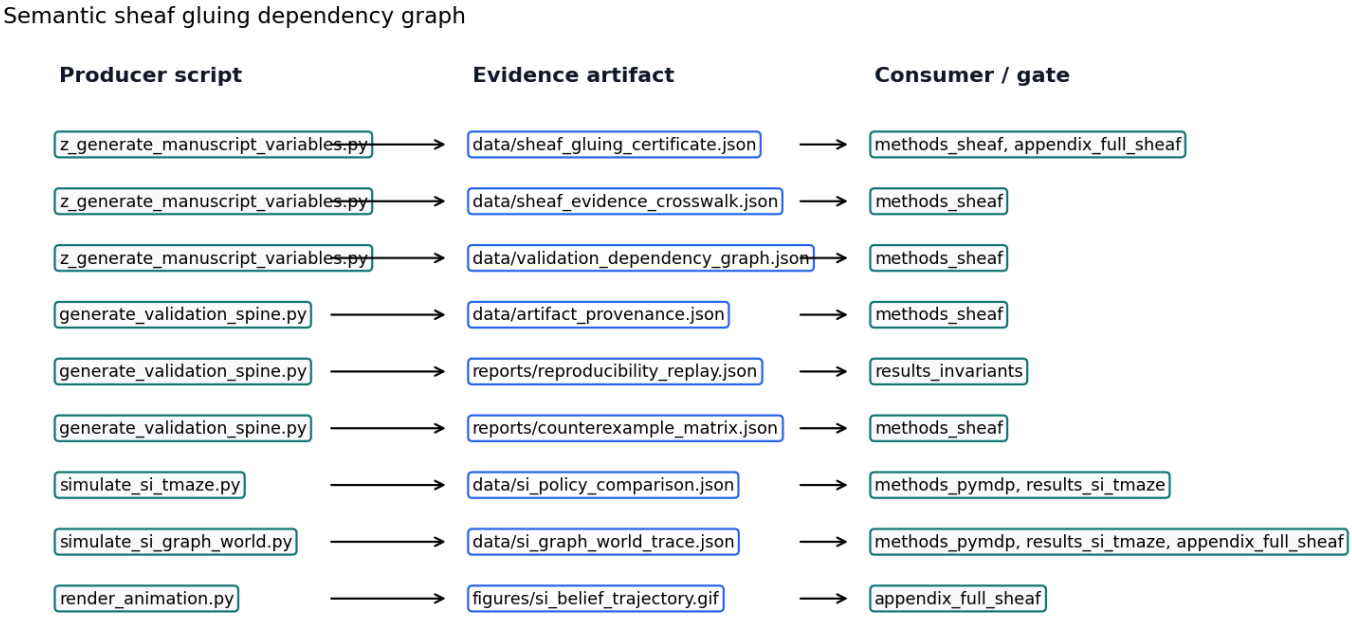

Figure 10: Dependency diagram linking configured analysis scripts to generated evidence artifacts, manuscript consumers, and validation gates for the semantic sheaf gluing certificate.

8.5 Sheaf fragment track registry

Compose order and renderer bindings from `manuscript/sheaf/tracks.yaml`.

Order	Track id	Label	Renderer	Optional
10	`prose`	Narrative prose	`markdown`	No
20	`formalism`	Mathematical formalism	`markdown`	No
30	`simulation`	Analytical simulation notes	`markdown`	No
35	`layers`	Sheaf layers tables	`layers_report`	Yes
40	`pymdp`	pymdp harness artifacts	`markdown`	No
42	`provenance`	Artifact provenance spine	`markdown`	No
44	`reproducibility`	Deterministic replay spine	`markdown`	No
46	`counterexample`	Expected-failure counterexamples	`markdown`	No
50	`visualization`	Figure references	`section_figures`	No
60	`lean`	Lean boundary fragment	`markdown`	No
70	`gnn`	GNN notation fragment	`markdown`	No
80	`ontology`	Active Inference Ontology bindings	`ontology_yaml`	No
90	`animation`	Animation fragment	`markdown`	Yes

Track count: 13 registered fragment types.

8.6 IMRAD binding matrix

Section rows versus fragment track columns. **P** = present (bound and file exists); — = absent (not bound); **M** = missing (bound, file absent).

Section	prose	formalism	simulation	layers	pymdp	provenance	reproducibility	counterexample	visualization	lean	gnn	ontology	animation
Introduction (group)	—	—	—	—	—	—	—	—	—	—	—	—	—
Motivation and scope	P	—	—	—	—	—	—	—	—	—	—	—	—
Contributions	P	—	—	—	—	—	—	—	P	—	—	P	—
Methods (group)	—	—	—	—	—	—	—	—	—	—	—	—	—
Bernoulli–Ising analytical model	P	P	P	—	—	—	—	—	P	—	P	P	—
pymdp simulation harness	P	P	—	—	P	—	—	—	P	—	P	P	—
Lean formalization boundary	P	—	—	—	—	—	—	—	P	P	—	—	—
Sheaf composition	P	P	—	P	—	P	—	P	P	—	—	—	—

Section	prose	formalism	simulation	layers	pymdp	provenance	reproducibility	interactivity	visualization	learning	gnn	ontology	animation
Results (group)	—	—	—	—	—	—	—	—	—	—	—	—	—
Mutual-information parameter sweep	P	P	P	—	—	—	—	—	P	—	—	—	—
Free-energy decomposition	P	—	—	—	—	—	—	—	P	—	—	—	—
T-maze active-inference rollout	P	—	—	—	P	—	—	—	P	—	—	—	—
Validation invariants	P	—	P	—	—	—	P	—	P	—	—	—	—
Discussion (group)	—	—	—	—	—	—	—	—	—	—	—	—	—
Limitations and outlook	P	—	P	—	—	—	—	—	—	—	—	P	—
Appendix (group)	—	—	—	—	—	—	—	—	—	—	—	—	—
Appendix: full track coverage	P	P	P	—	P	P	P	P	P	P	P	P	P

Totals: 53 present / 53 bound / 0 missing.

Symbol	Coverage color	Meaning
P	Black	Track **present** (bound and fragment exists)
—	White	**Absent** (not bound for this section)
M	Gray	**Missing** (bound but fragment file absent)

8.7 Evidence crosswalk

Claim	Artifact	Producer	Gates
`sheaf_registry`	`manuscript/sheaf/tracks.yaml`	`manual`	validate_outputs
`sheaf_manifest`	`manuscript/sheaf/manifest.yaml`	`manual`	validate_outputs
`sheaf_coverage_config`	`manuscript/sheaf/coverage.yaml`	`manual`	validate_outputs
`sheaf_coverage_matrix`	`output/data/sheaf_coverage_matrix.json`	`generate_figures.py`	validate_outputs
`sheaf_gluing_certificate`	`output/data/sheaf_gluing_certificate.json`	`z_generate_manuscript_variables.py`	validate_manuscript, validate_outputs
`sheaf_evidence_crosswalk`	`output/data/sheaf_evidence_crosswalk.json`	`z_generate_manuscript_variables.py`	validate_manuscript, validate_outputs
`validation_dependency_graph`	`output/data/validation_dependency_graph.json`	`z_generate_manuscript_variables.py`	validate_manuscript, validate_outputs
`semantic_gluing_graph_figure`	`../figures/semantic_gluing_graph.png`	`generate_figures.py`	validate_outputs, figure_registry

Claim rows: 30 typed evidence claims.

8.8 Artifact producer graph

Artifact	Producer	Configured	Consumers
output/data/analysis_statistics.json	compute_statistics.py	Yes	validate_outputs
output/data/artifact_provenance.json	generate_validation_spine.py	Yes	methods_sheaf
output/data/manuscript_variables.json	z_generate_manuscript_variables.py	Yes	validate_outputs
output/data/parameter_sweep.csv	run_analytical_sweep.py	Yes	validate_outputs
output/data/sheaf_coverage_matrix.json	generate_figures.py	Yes	validate_outputs
output/data/sheaf_evidence_crosswalk.json	z_generate_manuscript_variables.py	Yes	methods_sheaf
output/data/sheaf_gluing_certificate.json	z_generate_manuscript_variables.py	Yes	methods_sheaf, appendix_full_sheaf
output/data/si_graph_world_summary.json	simulate_si_graph_world.py	Yes	methods_pymdp, results_si_tmaze
output/data/si_graph_world_trace.json	simulate_si_graph_world.py	Yes	methods_pymdp, results_si_tmaze, appendix_full_sheaf
output/data/si_policy_comparison.json	simulate_si_tmaze.py	Yes	methods_pymdp, results_si_tmaze
output/data/si_tmaze_summary.json	simulate_si_tmaze.py	Yes	validate_outputs
output/data/si_tmaze_trace.json	simulate_si_tmaze.py	Yes	validate_outputs
output/data/validation_dependency_graph.json	z_generate_manuscript_variables.py	Yes	methods_sheaf
../figures/si_belief_trajectory.gif	render_animation.py	Yes	appendix_full_sheaf
output/reports/counterexample_matrix.json	generate_validation_spine.py	Yes	methods_sheaf
output/reports/invariants.json	run_analytical_sweep.py	Yes	validate_outputs
output/reports/reproducibility_replay.json	generate_validation_spine.py	Yes	results_invariants
output/reports/si_invariants.json	simulate_si_tmaze.py	Yes	validate_outputs
output/reports/si_tmaze_run_report.json	simulate_si_tmaze.py	Yes	validate_outputs

Producer issues: 0.

8.9 Semantic gluing restrictions

Restriction	Value
Coverage missing	0
Policy comparison rows	4
Graph-world trace agrees	True
Animation frames	4
Lean all proved	True
GNN ontology ok	True
Configured producers ok	True

Results

9 Mutual-information parameter sweep

We sweep coupling strength λ on a grid of 21 points up to $\lambda_{\max} = 4$. Closed-form mutual information from eq. 1 is compared to Monte Carlo estimates from the analytical module (sec. 5).

Measured invariant checks: 12 / 12 passed on the clean tree.

The sweep reuses the entangled joint defined in eq. 1 (sec. 5). Mutual information $I(\lambda) = \log 2 - H_b(\sigma(\lambda))$ is evaluated on the same λ grid as the analytical oracle and Monte Carlo estimates.

Empirical MI estimates use fixed seed 0 and share the same λ grid as the closed-form sweep (sec. 5, fig. 5).

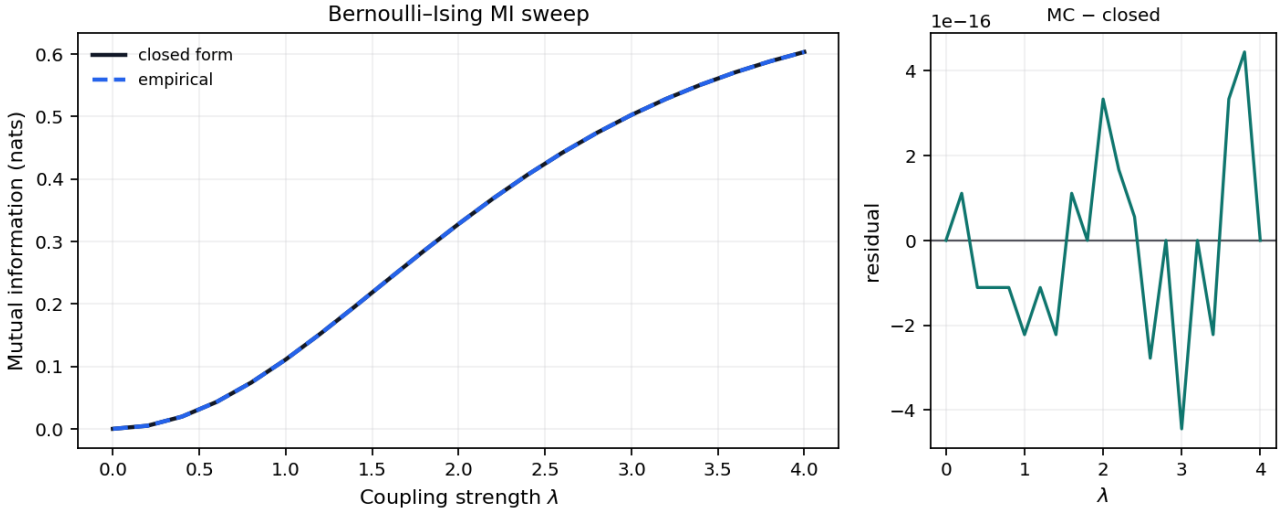

Figure 11: Two-panel plot of mutual information versus coupling strength lambda for the symmetric Bernoulli-Ising toy. Left panel shows closed-form curve (solid dark line) and Monte Carlo estimates (dashed blue line) with lambda on the x-axis and mutual information in nats on the y-axis. Right panel shows Monte Carlo minus closed-form residuals versus lambda with a zero reference line.

Figure 2 (results). Closed-form and Monte Carlo mutual information I(λ) for the symmetric Bernoulli-Ising toy across 21 grid points up to λ_max = 4; grid maximum 0.6031 nats on the measured sweep.

10 Free-energy decomposition

Free energy against the entangled prior is evaluated along the same λ grid used for the MI sweep. The curve highlights how deformation strength trades off prior alignment and coupling energy in the analytical toy.

Saturation MI (grid maximum on the measured λ sweep): 0.6031 nats.

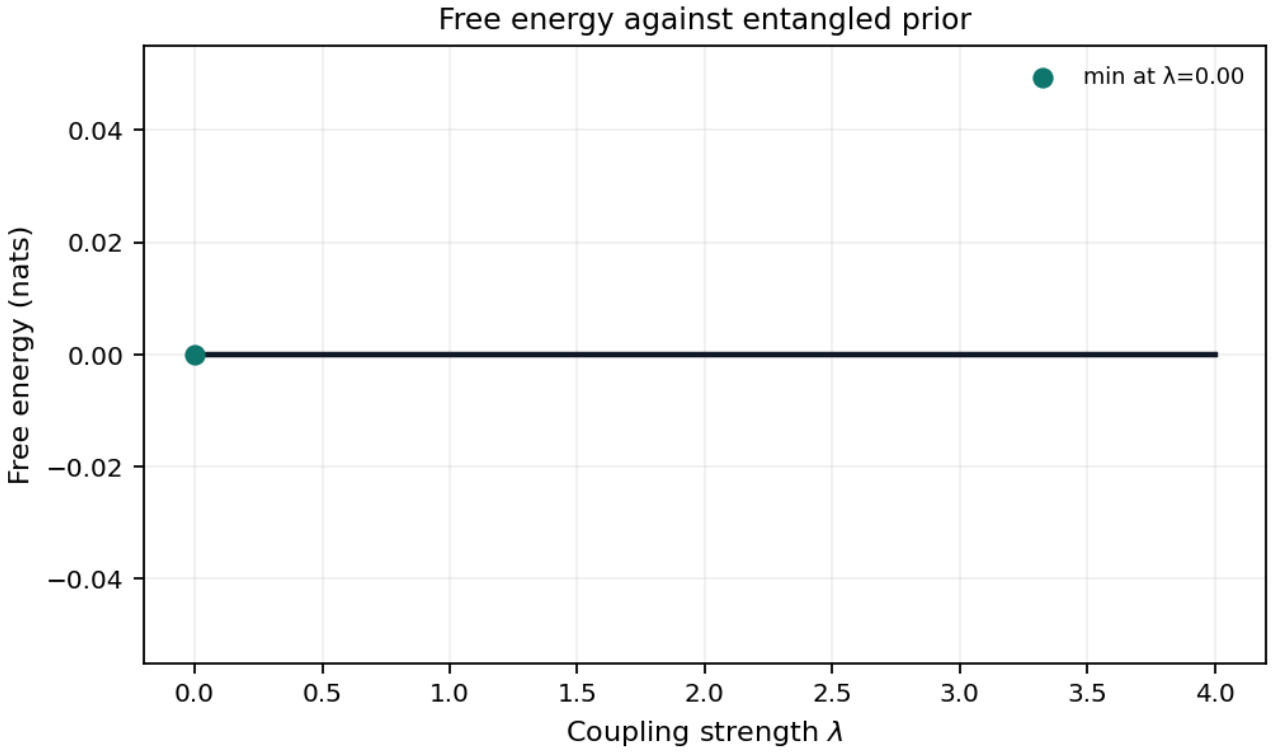

Figure 12: Line plot of free energy in nats versus coupling strength lambda for the entangled posterior relative to a mean-field prior. A single dark curve traces free energy across the sweep; the minimum is marked with a teal scatter point and labeled with the argmin lambda value.

Figure 3 (results). Free energy of the entangled posterior relative to the mean-field prior across the hyperparameter sweep (grid points 21).

11 T-maze active-inference rollout

The pymdp harness rolls out a T-maze active-inference agent in `state_inference` mode with planning horizon 2. The default `state_inference` mode is belief filtering with a goal-seeking action rule; sophisticated policy inference (an expected-free-energy policy posterior) is selectable via `mode: policy_inference` (sec. 6). Summary metrics land in `output/data/si_tmaze_summary.json`.

Steps recorded: 2. Mean belief entropy: 0.3251.

Policy-comparison rows: 4 across state-inference and policy-inference modes; goal-reaching rows: 3. Graph-world extension rows: 4 over 4 nodes, with goal-reached flag 1.

Rollout trace: `output/data/si_tmaze_trace.json`. JSONL run log: `output/logs/pymdp_runs.jsonl`.

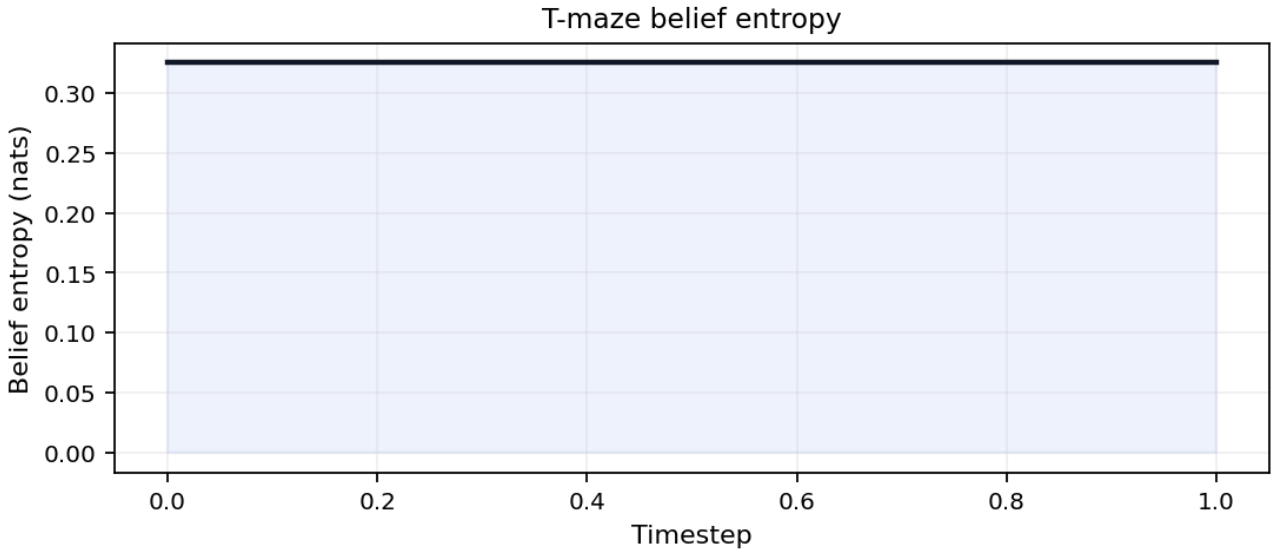

Figure 13: Line plot of belief entropy in nats versus timestep for the pymdp T-maze rollout. Entropy ranges from 0.3251 to 0.3251 nats across 2 steps in state_inference mode.

Figure 3a (results). Belief entropy over time for the T-maze rollout (mean 0.3251 nats).
Figure 3b (results). Observation and action traces for the T-maze rollout (action diversity 2).
Figure 3c (results). Discrete action index over time for the pymdp T-maze rollout (policy length 2).

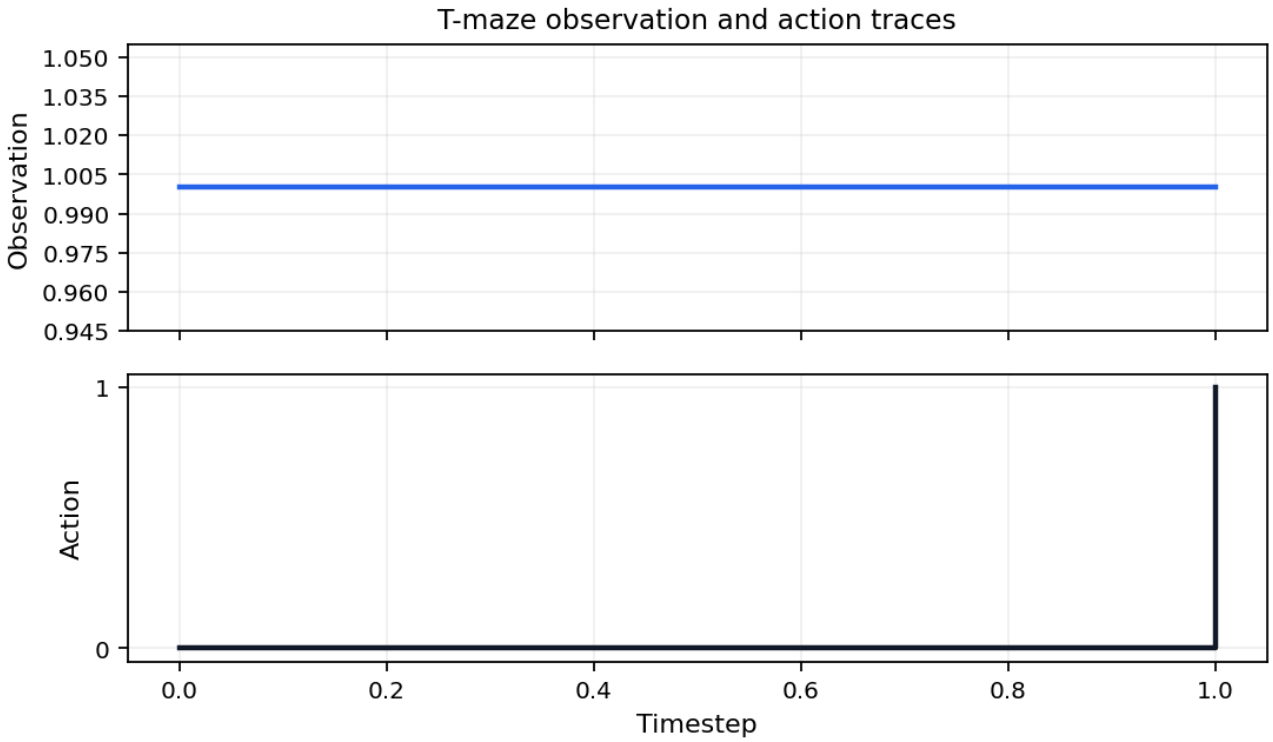

Figure 14: Dual-panel plot of observation index and action index versus timestep for the pymdp T-maze rollout. The upper panel shows discrete observations; the lower panel shows actions. Goal reached flag is 1.

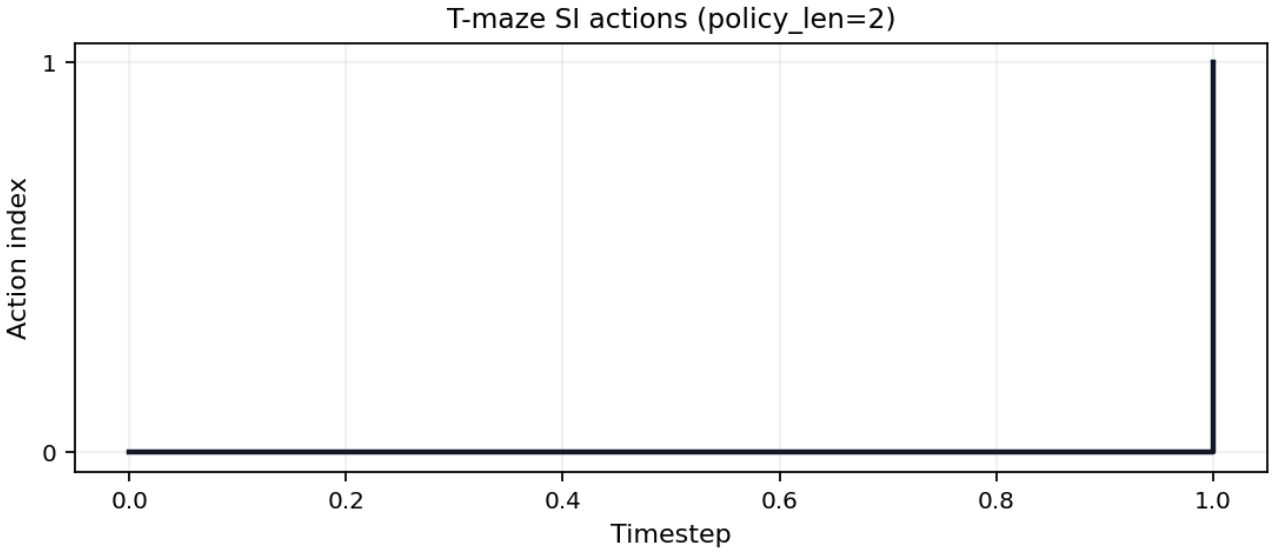

Figure 15: Step plot of discrete action index versus timestep for the pymdp T-maze rollout in state_inference mode. Actions change at each timestep with light fill under the step trace; policy depth is 2 steps.

12 Validation invariants

The analytical invariant registry runs before PDF rendering (sec. 5). On a clean checkout **12 / 12** checks pass in the merged validation report, which records simulation invariants when the pymdp harness ran (sec. 11).

fig. 16 lists each analytical and simulation gate; failures block publication artifacts. See sec. 8 for how invariant counts hydrate manuscript tokens.

Simulation invariants merge into the analytical report after the pymdp harness runs (sec. 11). fig. 16 summarizes pass/fail status for both domains on the clean tree.

The `reproducibility` fragment replays deterministic toy producers in a temporary project tree and compares regenerated outputs with the saved artifacts. The current replay report records 5 checks and an all-passed flag of 1.

The replay scope is deliberately narrow: analytical parameter sweeps, graph-world summary/trace artifacts, and the policy-comparison table. It does not claim platform-independent bitwise equivalence for PDF rendering, PNG rasterization, or external empirical data.

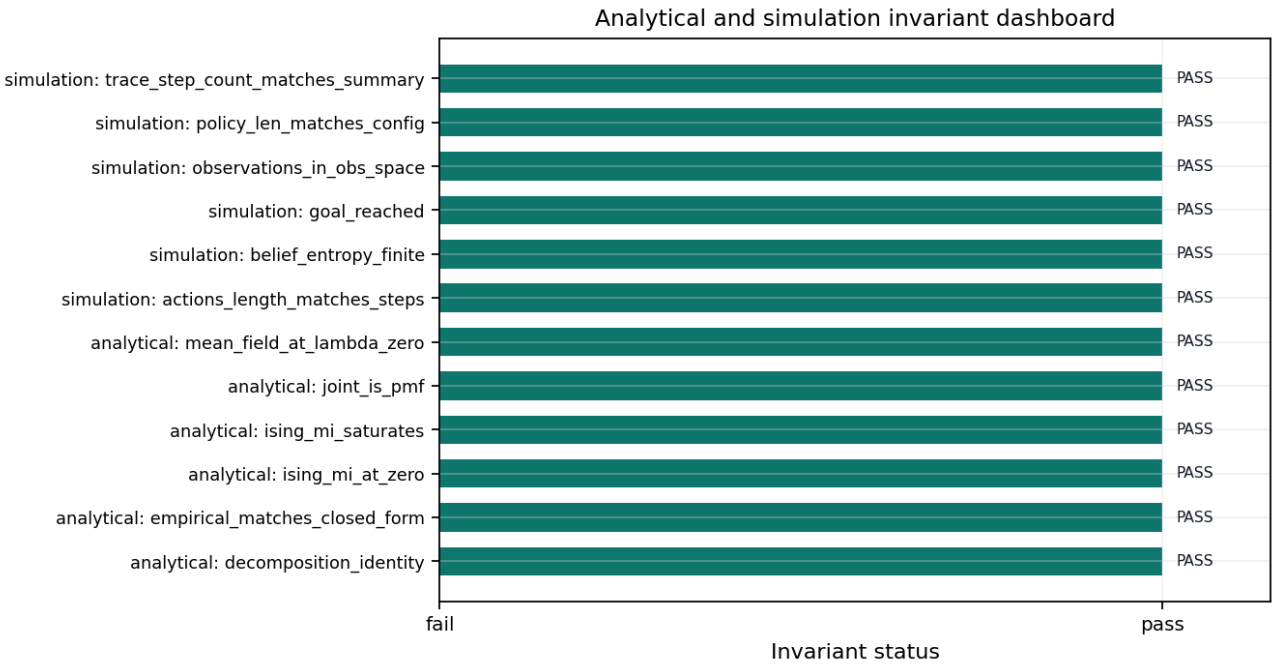

Figure 16: Horizontal bar checklist of analytical and simulation invariant names with pass or fail status. 12 of 12 checks pass on the merged report.

Figure 5 (results). Invariant dashboard: 12 / 12 merged analytical and simulation checks from the validation registry.

Discussion

13 Limitations and outlook

13.1 Limitations

The Bernoulli–Ising toy, T-maze harness, and sheaf composition model are pedagogical. They validate analytical consistency, artifact wiring, renderer dispatch, and manuscript hydration—not empirical claims about biological agents. Default pymdp mode is `state_inference` with planning horizon 2; the policy-comparison artifact exposes policy-inference rows without changing the default rollout (sec. 6).

13.2 Sheaf audit and outlook

sec. 1 and sec. 14 make binding state auditable under strict compose validation (sec. 8). Pipeline extensions in `tracks.yaml` `extension_tracks` now write deterministic artifacts: a belief GIF via `render_animation.py` and graph-world SI summary/trace via `simulate_si_graph_world.py`. The appendix row already binds an `animation` sheaf fragment without new manifest rows.

Sweep RMSE 0 nats and SI goal reached 1 summarize measured agreement on the declared grids and rollout. Future work includes full expected-free-energy policy selection, richer graph-world rollouts, and expanded Lean proofs beyond the boundary witnesses in sec. 7.

The discussion ontology binds `coverage_semantics` to the audit matrix in sec. 1, `pedagogical_scope` to the non-empirical scope of the toy models, and `state_inference_mode` to the pymdp harness contract in sec. 6.

Measured pymdp rollout (`state_inference`, config hash `ea4b126a9c22f22f`): mean belief entropy 0.3251 nats over 2 steps; goal reached flag 1; action diversity 2.

Analytical sweep residual RMSE 0 nats (max residual 0). Coverage audit: 53 present / 53 bound / 0 missing cells on the IMRAD matrix.

13.2.1 Ontology bindings

- `coverage_semantics` → **Coverage matrix semantics**
- `pedagogical_scope` → **Pedagogical scope**
- `state_inference_mode` → **State inference harness**

Appendix

14 Appendix: full track coverage

This section is the **composability proof** for the manifest-indexed sheaf model: all 12 appendix-bound fragment tracks render into one flat manuscript section without section-specific compose branches. The registry defines 13 composable types; optional `layers` is methods-only and excluded from this row. The `animation` fragment is bound here as an optional registry type alongside prose, formalism, simulation, pymdp, provenance, reproducibility, counterexample, visualization, lean, gnn, and ontology tracks.

The proof is a publication-systems check (eq. 4). It demonstrates that heterogeneous fragments share one registry, manifest, renderer dispatch path, coverage matrix, and hydration boundary; it does not assert that every track carries equal scientific weight.

For each track $t \in \mathcal{T}_{\text{Full}}$, the appendix row binds a fragment path $f(t)$ and the composer emits `<!-- sheaf-track:t -->` before the rendered body. Generated renderers such as `section_figures` and markdown renderers pass through the same `resolve_track_body()` dispatch, so the appendix exercises the common compose interface rather than a bespoke appendix path.

$$|\mathcal{T}_{\text{Full}}| = 12 \tag{4}$$

The fragment registry defines 13 composable track types; optional `layers` is bound on the methods sheaf section only. Optional `animation` is bound in this appendix proof; the deterministic GIF artifact in `tracks.yaml extension_tracks` is produced by the core analysis DAG and remains separate from this fragment slot.

Because this appendix binds every non-optional track plus both optional tracks, it is the maximal stalk of the coverage presheaf and exercises every renderer through the common `resolve_track_body()` dispatch. The same compose path is gated by the 6 sheaf laws verified in sec. 8 (6/6 satisfied): the appendix section glues to a unique output (separation), occupies the terminal position of the linear extension under its own `appendix` group row (poset and gluing), binds only well-typed fragments (typing), and owns every fragment path it references (compositionality). No count in this appendix is hand-written; all are injected from the registry-backed oracle.

Analytical sweep artifacts feed sec. 9 and sec. 12; simulation invariants merge after sec. 11. No additional path listing is required beyond those Results sections.

pymdp harness summary: `output/data/si_tmaze_summary.json` (mean belief entropy, action trace). Full log: `output/logs/pymdp_runs.jsonl`.

The appendix provenance fragment points to `output/data/artifact_provenance.json`, the promoted validation-spine artifact that records core artifact hashes, producer scripts, and config digests.

The appendix reproducibility fragment points to `output/reports/reproducibility_replay.json`, which replays deterministic toy producers in a temporary tree and compares regenerated artifacts to the saved outputs.

The appendix counterexample fragment points to `output/reports/counterexample_matrix.json`, the expected-failure matrix that keeps promoted validation gates falsifiable.

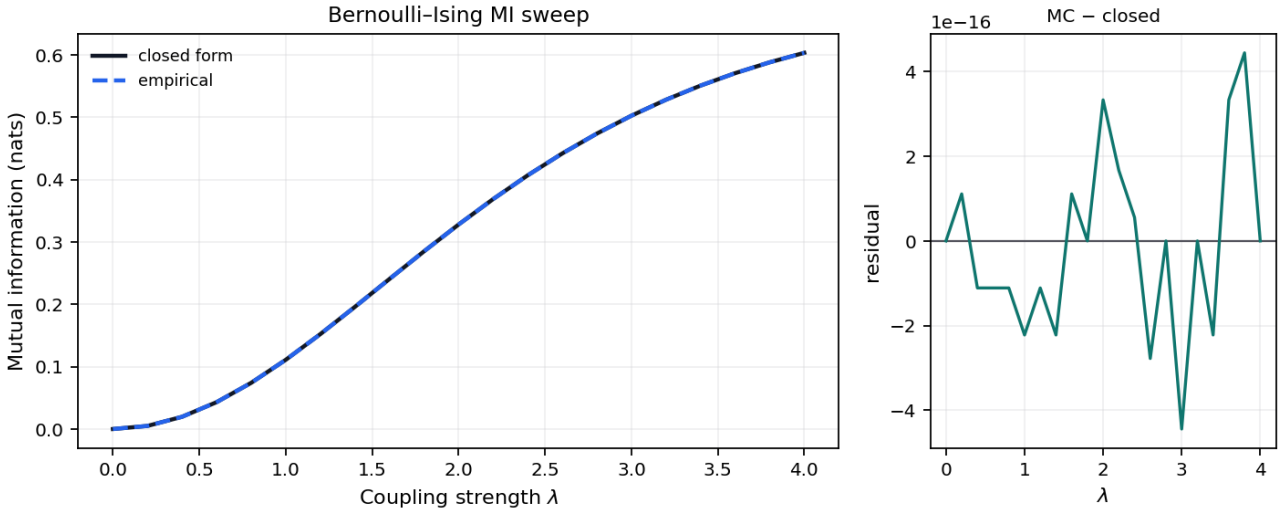

Figure 17: Two-panel plot of mutual information versus coupling strength lambda for the symmetric Bernoulli-Ising toy. Left panel shows closed-form curve (solid dark line) and Monte Carlo estimates (dashed blue line) with lambda on the x-axis and mutual information in nats on the y-axis. Right panel shows Monte Carlo minus closed-form residuals versus lambda with a zero reference line.

Figure A1 (appendix). Closed-form and Monte Carlo mutual information I(λ) for the symmetric Bernoulli-Ising toy across 21 grid points up to λ_max = 4; grid maximum 0.6031 nats on the measured sweep.

Figure A2 (appendix). Discrete action index over time for the pymdp T-maze rollout (policy length 2).

Figure 4. Sheaf track coverage matrix: 17 IMRAD rows × 13 fragment columns. Black = present (P), white = absent (—), gray = missing (M). Counts: 53 present / 53 bound / 0 missing.

Lean modules under `lean/TemplateActiveInference/` declare horizon and coupling witnesses. Build with `lake build` in `lean/`; fig. 8 summarizes proved versus deferred statements for this boundary fragment.

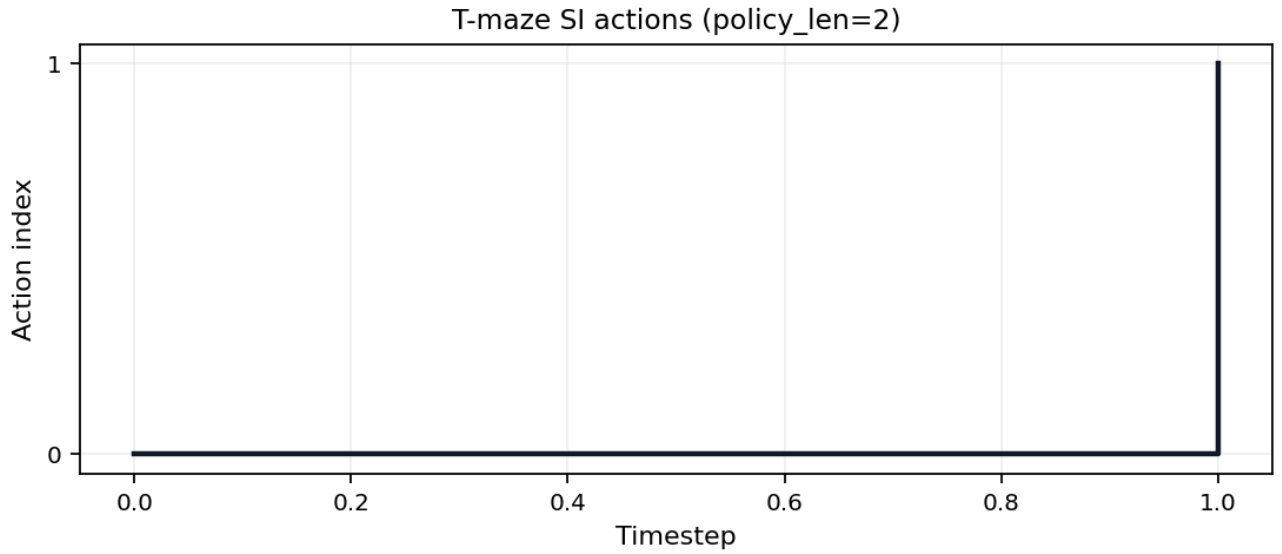

Figure 18: Step plot of discrete action index versus timestep for the pymdp T-maze rollout in state_inference mode. Actions change at each timestep with light fill under the step trace; policy depth is 2 steps.

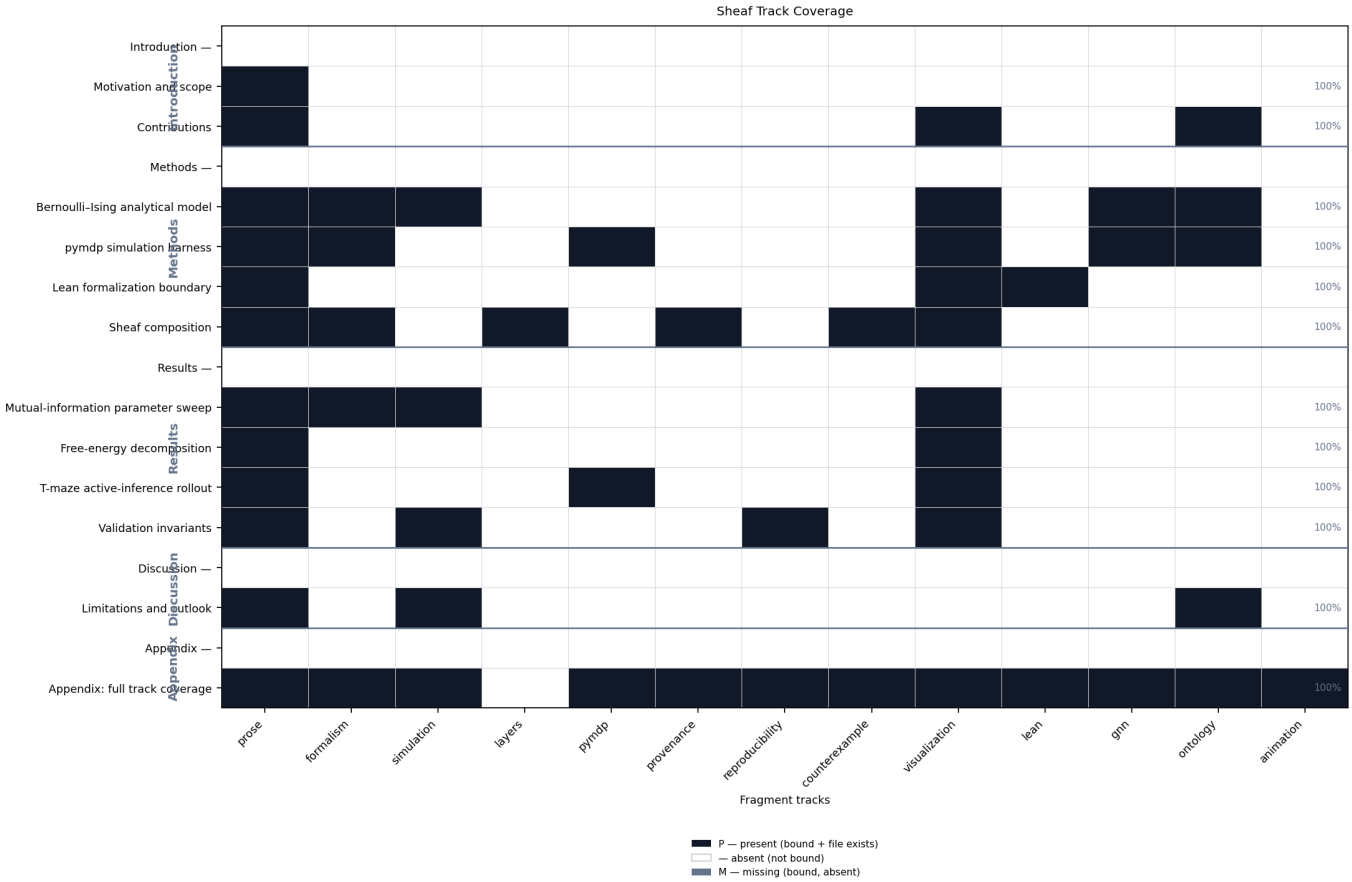

Figure 19: Heatmap matrix of IMRAD manuscript rows versus 13 sheaf fragment track columns. Black cells mean the track is bound and the fragment file exists; white cells mean the track is not bound; gray cells mean bound but missing. Rows are grouped by IMRAD block with indented subsection labels; column headers list track ids.

GNN declarations: `gnn/bernoulli_toy.gnn.md` and `gnn/si_tmaze.gnn.md`. fig. 6 and sec. 5 document ontology concordance for the Bernoulli toy; SI notation extends the same pattern under sec. 6.

14.0.1 Ontology bindings

- `belief_entropy` → **BeliefEntropy**
- `expected_free_energy` → **ExpectedFreeEnergy**
- `location` → **HiddenState**
- `observation` → **ObservationLikelihood**
- `policy` → **PolicyPosterior**
- `sheaf_track` → **SheafFragment**

Animation is an **extension** sheaf track backed by a deterministic GIF from `scripts/render_animation.py`. This appendix row documents the track binding only; default publication still uses static SI figures (sec. 11, fig. 15) while the GIF remains an auditable generated artifact.

15 Conclusion

Analytical oracles (sec. 5), pymdp rollouts (sec. 11), and sheaf composition (sec. 8) share one auditable manuscript contract: measured artifacts hydrate 12 composed sections, sec. 1 reports binding state, and strict compose validation blocks gray matrix cells before PDF rendering.

The T-maze harness runs in `state_inference` mode with config hash `ea4b126a9c22f22f`; sweep RMSE 0 nats summarizes analytical–empirical agreement on the toy coupling grid. sec. 12 merges analytical and simulation gates; sec. 13 states scope and extensions. Scientific claims remain confined to declared models—not empirical statements about biological agents.

16 References

See `manuscript/references.bib` for bibliography entries cited in the composed sections.

References

END OF TRANSMISSION

Release: v0.3.0 · DOI 10.5281/zenodo.20417021 · SHA-256 pending… · pairing pending

Figure 20: Integrity QR strip

Prior: *No prior releases.*